OUTPUT:

```
35
['\n', ' ', ',', '.', 'A', 'D', 'L', 'M', 'N', 'R', 'T', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y']
tensor([ 0,  4, 27, 29, 19, 16, 19, 13, 19, 11, 21,  1, 19, 23, 29, 15, 21, 21,
        19, 17])
tensor([ 0,  4, 27, 29, 19, 16, 19, 13, 19, 11, 21,  1, 19, 23, 29, 15, 21, 21,
        19, 17, 15, 23, 13, 15,  1, 19, 28,  1, 13, 18, 11, 23, 17, 19, 23, 17,
         1, 29, 18, 15])
tensor(1)
```

```
Epoch: 0, Loss: 3.5495803356170654
Epoch: 1, Loss: 3.49232816696167
Epoch: 2, Loss: 3.432427167892456
Epoch: 3, Loss: 3.331331968307495
Epoch: 4, Loss: 3.1706511974334717
Epoch: 5, Loss: 3.089303731918335
Epoch: 6, Loss: 3.046177864074707
Epoch: 7, Loss: 3.034759759902954
Epoch: 8, Loss: 3.0309834480285645
Epoch: 9, Loss: 3.021061897277832
Epoch: 10, Loss: 3.010931968688965
Epoch: 11, Loss: 3.004021406173706
Epoch: 12, Loss: 2.995473623275757
Epoch: 13, Loss: 2.983480215072632
Epoch: 14, Loss: 2.9705710411071777
Epoch: 15, Loss: 2.958589553833008
Epoch: 16, Loss: 2.947445869445801
Epoch: 17, Loss: 2.9363205432891846
Epoch: 18, Loss: 2.9247629642486572
Epoch: 19, Loss: 2.912441492080688
```

```
Machine learning helps computers learn ovyleeinrrsprnp nlwcninit.satpefg o
 eieaeedtrus ssteennraiuh gp.sterre imupi iRgaets mhtntiinwo n.cclue  nRavsnnig otcolrnll  tarsere enalenledee eeqlcwaedts
uxd bacrnrecenone
vxleun ett recrAceorsnueeeaicrno ter
```

Code:

```python
import torch
import torch.nn as nn

text = """
Artificial intelligence is changing the world.
Machine learning helps computers learn from data.
Deep learning uses neural networks to solve complex problems.
Recurrent neural networks process sequential information.
They remember information from previous steps using hidden states.
Natural language processing helps machines understand human language.
Text generation predicts the next character based on previous characters.
Learning requires data, patience, and repeated experiments.
Technology can help people solve important problems.
Researchers continuously improve intelligent systems.
""" * 20

chars= sorted(set(text))
char_to_id = {ch: i for i , ch in enumerate(chars)}
id_to_char = {i: ch for i , ch in enumerate(chars)}

vocab_len = len(chars)
print(vocab_len)
print(chars)

encoded = torch.tensor([char_to_id[ch] for ch in text])
print(encoded[:20])

seq_length = 40

X = [] #Input
Y = [] #output

for i in range(0, len(encoded) - seq_length):
    X.append(encoded[i : i + seq_length])
    Y.append(encoded[i + seq_length])
X = torch.stack(X)
Y = torch.stack(Y)
print(X[0])
print(Y[0])

X = nn.functional.one_hot(X, num_classes=vocab_len).float()
Y = nn.functional.one_hot(Y, num_classes=vocab_len).float()

class RNNText(nn.Module):
  def __init__(self, vocab_len, hidden_size):
    super().__init__()
    self.hidden_size = hidden_size
    self.rnn = nn.RNN(vocab_len, hidden_size, batch_first=True)
    self.fc = nn.Linear(hidden_size, vocab_len)

  def forward(self, X):
    out, _ =self.rnn(X)
    out = self.fc(out)
    return out

model = RNNText(vocab_len, 128)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.003)

target_indices = torch.argmax(Y, dim=1).long() if Y.dim() > 1 else Y.long()
for epoch in range(20):
  prediction= model(X)

  loss = criterion(prediction[:, -1, :], target_indices)

  optimizer.zero_grad()
  loss.backward()
  optimizer.step()

  print(f"Epoch: {epoch}, Loss: {loss.item()}")

def generate_text(model, seed, length=200, temperature=0.8):
    model.eval()
    generated = seed

    with torch.no_grad():
        for _ in range(length):

            context = generated[-seq_length:]
            idx = torch.tensor([[char_to_id[ch] for ch in context]])
            x = nn.functional.one_hot(idx, num_classes=vocab_len).float()

            output = model(x)
            logits = output[:, -1, :] / temperature
            probs = torch.softmax(logits, dim=-1)

            next_id = torch.multinomial(probs, num_samples=1).item()
            generated += id_to_char[next_id]

    return generated
seed = "Machine learning helps computers learn "
print(generate_text(model, seed, length=300))
```